

以 I3DM 复现项目为例： 编程能力 · Debug 能力 · Debug 思路

项目：Implicit 3D-aware Memory Retrieval and Injection (I3DM)

平台：中国科学技术大学本科教学算力平台 · 单卡 A100 80GB · Slurm

一、最能体现编程与 Debug 能力的项目

我选择的项目是：在本科生算力配额（1 GPU / 4 CPU / 单作业 4 小时）下，复现长视频 3D 一致生成方法 I3DM。工作覆盖两条主线——推理评测（Re10K 上 32 个场景产出完整漫游视频）与冒烟训练（LoRA 200 步，验证前向/反向与 checkpoint 落盘）。它比单纯「跑通 demo」更能体现能力：需要数据管线、集群调度适配、以及一次有对照实验的显存根因定位。

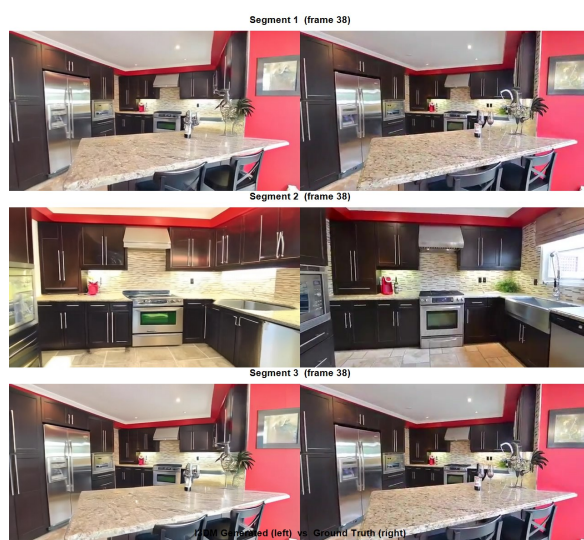


图 1 场景 005dd9a58df1ba3c：左列 I3DM 生成 / 右列真值（取第 1、3、6 段中间帧）。单张参考图驱动约 28 秒自回归漫游，跨段视角变化下厨房几何与外观保持一致——这是推理管道跑通的直观证据。

复现边界（如实）：未重写网络结构；未跑满论文 11000 步训练；官方 eval 不聚合 FVD/PSNR，本轮以管道正确性与定性一致性为验收标准。训练侧唯一偏离论文设定的改动是显存约束下的帧数（77→41）。

二、哪些部分体现了编程能力

编程能力主要体现在：读懂他人训练/评测代码，把数据、作业与模型接口收成可在配额内重复执行的管线，而不是一次性手动点跑。

1. 数据管线

根据 Re10K 的场景→文件映射按需下载（相对全量约省 78%），解包后切出 530 个 77 帧训练片段，并对 200 个样本做张量形状预检（视频 + 输入视图 + 目标视图 + 参考图），保证训练入口格式正确。

2. 评测工程与断点续跑

单场景约 13–14 分钟，4 小时墙钟装不下大批量任务。我将场景列表分块，并为评测脚本增加：若已存在 video_combined.mp4 则跳过——作业可接力、可恢复，最终 32/32 场景零失败产出。

3. 训练接入与版本锁定

锁定 torch 2.5.0+cu124 □ xformers 0.0.28.post2 □ transformers 4.57.3, 配置 DiT LoRA (rank 1024) 与可训练模块 ref_conv / nvs_model, 用 sbatch + accelerate 提交可复现的冒烟训练作业。

4. 约束驱动的系统设计

配额是方案的一部分：1 卡、4 核、单作业 4 小时。完整 11000 步经评估放弃；改为 200 步冒烟验证管道健康度——这是工程判断，也是编程落地时的取舍能力。

```
评测续跑补丁（核心逻辑）
if os.path.exists(.../video_combined.mp4): # 已完成则跳过
    print('==SKIP done==', scene); continue

训练作业关键配置
LoRA rank=1024 · trainable: ref_conv, nvs_model · save_steps=50 · wandb offline
```

三、哪些部分体现了 Debug 能力

Debug 能力集中体现在训练阶段的连续排错，尤其是一次有假说、有对照、能否定错误原因的显存定位。

3.1 训练冒烟：五次提交中的定位过程

作业	现象	动作与结论
52883	缺 pandas	补装 2.3.3
53053	缺 easydict	扫描 import, 一次性补齐隐式依赖
53054	缺 wandb → 随后 CUDA OOM (77 帧前向 ≈78.5 GiB)	补装 wandb; 开始显存剖析
53055	rank 1024→512 仍 OOM (显存几乎不变)	否定「LoRA 过大」假说：瓶颈是帧数激活，非 rank
53057 □	改 num_target_views 77→41 (满足 VAE 时间维整除)	rank 仍 1024, 一次通过 200 步 / 42 分钟

表 1 冒烟训练排错记录。关键转折：降 LoRA rank 不能降低显存 → 证明激活由序列帧数驱动 → 改数据集目标帧数为 41。

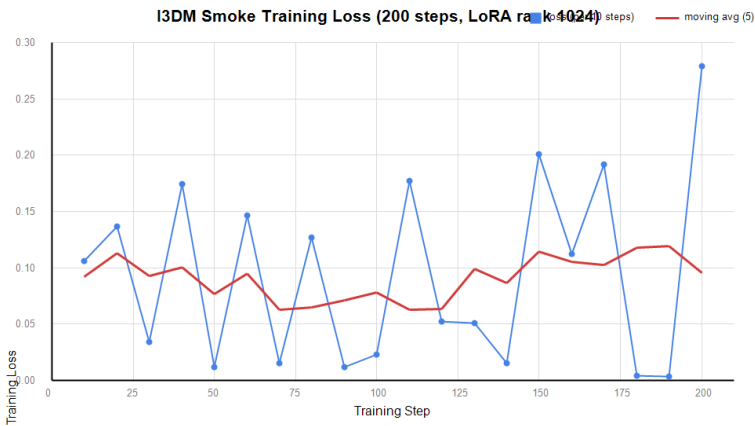


图 2 200 步冒烟训练 loss (蓝：原始；红：5 点滑动平均)。单步 loss 因扩散 timestep 采样而天然高噪声；验收标准是无 NaN、无发散、梯度稳定、checkpoint 可落盘——而非要求 200 步内出现论文级平滑下降。

3.2 其他典型 Debug 实例

- 路径类：首次冒烟评测把数据放在登录节点 /tmp，计算节点不可见而失败；改为共享主目录后解决。
- 依赖类：官方 requirements 不宜直接全量安装；训练脚本隐式依赖逐个爆雷后，改为提交前静态扫描 import。
- 调度类：单作业 4 小时装不下批量评测 → 分块 + 续跑，而不是硬扛超时。

四、一般的 Debug 思路

我把 Debug 当成小型科学实验：先让失败可重复，再提出假说，用最小对照检验；假说被否定就换方向，而不是同时改多处「碰运气」。这与系统化调试中常见的「观察 → 假说 → 预测 → 实验 → 修正」一致。结合 I3DM 的 OOM 案例，步骤如下：

① 稳定复现

把问题收成最小可重复案例（单场景评测、短步数训练）。复现不了，就不急着改代码。

② 分层隔离

先判断故障落在：环境与依赖 / 数据与路径 / 模型前向与显存 / 作业调度。每次只动一层。

③ 假说 + 对照（核心）

写下「若原因是 X，则改 X 后现象应如何变」。OOM 时假说是「LoRA 过大」；预测是降 rank 后显存明显下降；实验后显存几乎不变 → 否定假说 → 转向帧数。

④ 最小修改 + 冒烟验证

只改必要处（帧数与 VAE 整除约束），先用 200 步确认通过，再考虑放大规模。

⑤ 回归与留痕

确认原问题消失；记录有效配置与失败作业号，避免同类问题重复踩坑。

一句话：Debug 的目标不是「改到能跑」，而是「用对照实验证明自己找对了原因」。I3DM 冒烟训练里，降 rank 那一步的价值不在于它成功了——而在于它失败得干净，把真因锁到了帧数。

附图与数据来自本次复现实证目录 results_showcase/ · 完整 32 场景结果存于算力平台